Discover Computing

Research

SignEdgeLVM transformer model for enhanced sign language translation on edge devices

Rina Damdoo[1] · **Praveen Kumar**[2]

Received: 9 October 2024 / Accepted: 25 February 2025
Published online: 10 March 2025
© The Author(s) 2025 OPEN

Abstract
Transformer architectures have accelerated the research in Continuous Sign Language Recognition and Translation (CSLRT), which involves predicting sign gloss patterns from video and converting them into spoken language. This process is challenging due to the lack of direct alignment between sign glosses and spoken words. While Transformers are effective due to their ability to process inputs in parallel, their high memory consumption makes the transformers less suitable for edge devices. To address this issue, we propose the SignEdgeLVM, a model that uses a Global Relative Attention Matrix (GRAM) and Dynamic Point Frame Sampling (DPFS) module. The SignEdgeLVM significantly lowers attention mechanism memory consumption by 78.22 MB (99.93 percent) per head in the attention layer for our implementation. Evaluated on the PHOENIX14T dataset, this optimization makes SignEdgeLVM suitable for edge devices.

Keywords Sign language recognition and translation · Deep learning · Sign language transformer · Global relative attention matrix · Dynamic point frame sampling · PHOENIX14T dataset · SignEdgeLVM

1 Introduction

Sign language is the natural way of conversation commonly used by humans who are mute or deaf. The gestures or symbols in sign language called signs; are organized linguistically. As reported by the World Federation of the Deaf, there are 72 million deaf people worldwide, and 80 percent of them reside in developing nations. Because sign languages differ from country to country, more than 300 different sign languages are in use worldwide. This is due to each language's unique vocabulary, grammar, and linguistic rules. Even in countries where the same language is spoken, sign language can have a diverse variety of regional inflections that bring distinct variances to the way people use and comprehend signs. The Ethnologue language database, a comprehensive reference source collecting the world's known living languages lists 130 sign languages [1].

The objective of sign language translation is to automatically generate representations from sign videos, either by transcribing them into a list of transcription of sign sequences known as glosses, where the individual sign has its gloss, or into a sequence of spoken language words, or jointly into both glosses and spoken language words, as illustrated in Fig. 1 Sign Production i.e. generation of human-like non-manual sign videos (3D animation) from spoken language textual information [3, 4] is another challenge for investigators in the domain of computer vision and machine learning to help speech and hearing-impaired people. Since there is a difference in spoken and sign language syntax, straightforward sign-to-word mapping is challenging, and converting sign language videos into spoken language sentences

✉ Rina Damdoo, damdoor@rknec.edu; Praveen Kumar, praveenkumar@cse.vnit.ac.in | [1]Shri Ramdeobaba College of Engineering and Management, Ramdeobaba University, Nagpur 440013, India. [2]Visvesvaraya National Institute of Technology, Nagpur 440010, India.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

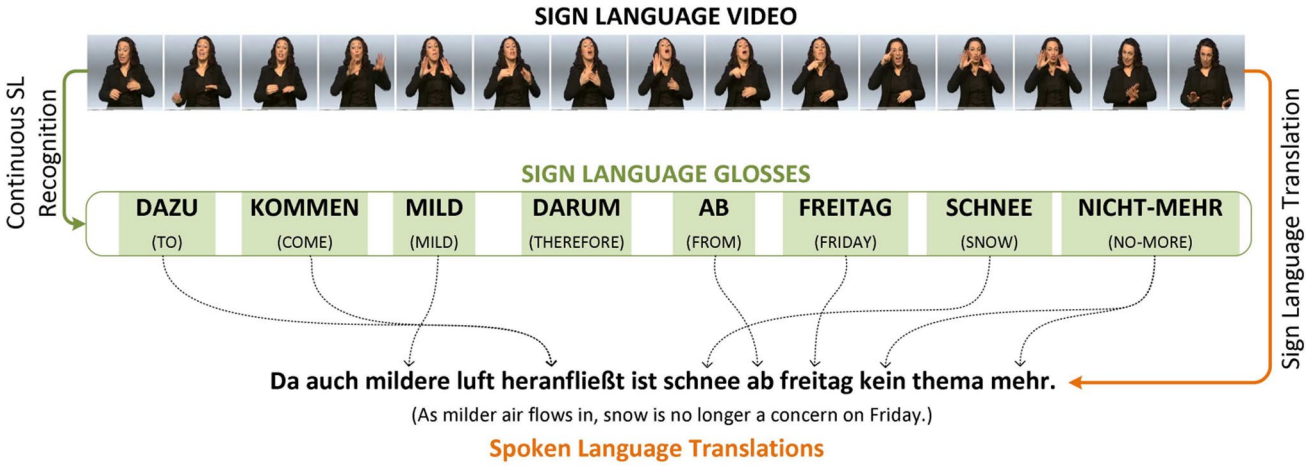


Fig. 1 Continuous sign language recognition and sign language translation [2] https://www-i6.informatik.rwth-aachen.de/~koller/RWTH-PHOENIX-2014-T/

is a spatiotemporal machine translation task [3]. Although Transformers have significantly improved performance in computer vision, natural language processing, and Continuous Sign Language Recognition and Translation (CSLRT) by processing parallel data, their high computational and memory demands, due to a large number of parameters remain a challenge. Memory utilization and optimization in these models is still relatively under-explored. As transformers are unaware of the relative positions of frames in sign sequence, few studies including [5–8] can be found which investigate the issue and use relative position embeddings for the transformers, but do not focus on the computational and memory demands, limiting sign language transformer's application on edge devices such as mobile phones and embedded systems. Edge computing devices play a crucial role in enhancing communication for deaf individuals by supporting real-time sign language translation and speech-to-text applications. By processing data locally, these devices offer significant benefits in daily interactions, ensuring greater privacy and security while allowing for independent use in areas with limited connectivity. Additionally, edge devices provide affordable, portable solutions that can be integrated into wearables, empowering deaf individuals with more autonomy and reducing their dependence on external services or infrastructure. Considering the limitations of existing methods, we propose a model SignEdgeLVM in this paper that uses a Global Relative Attention Matrix (GRAM) and Dynamic Point Frame Sampling (DPFS) module. The SignEdgeLVM significantly lowers attention mechanism memory consumption by 78.22 MB (99.93 percent) per head in the attention layer for our implementation. We also introduce Dynamic Point Frame Sampling (DPFS) to reduce the computational demands. This frame selection module intelligently picks informative frames from a sign video sequence instead of processing every frame reducing the data to be processed, and concentrating on key segments of the video. Our work aims to exploit the transformer's strength in CSLRT using computational and memory-efficient implementation. The proposed work is the first to optimize the computation and memory needed by the complex architecture of the sign language transformer. Experimental results using the PHOENIX14T dataset [2] demonstrate the effectiveness of the proposed method and make the SignEdgeLVM model suitable for edge devices. The main contributions of this work can be summarized as follows:

1. We introduce the SignEdgeLVM model for sign language translation that incorporates a Global Relative Attention Matrix (GRAM) across all heads in a transformer layer, significantly reducing memory requirements by 99.93 percent per attention layer.
2. We introduce a frame selection module, Dynamic Point Frame Sampling (DPFS), which intelligently selects informative frames from a sign video sequence, reducing computational demands and focusing on key segments of the video.
3. The proposed method makes the SignEdgeLVM model, suitable for deployment on edge devices.

The remainder of this paper is structured as follows: Sect. 2 reviews the prior research on Sign Language Translation (SLT) and the current state of the art in Neural Machine Translation (NMT). Section 3 presents the transformer and its operations. In section 4, the SignEdgeLVM is proposed, and the implementation details are provided. The baseline findings for the proposed SignEdgeLVM are discussed in section 5. Section 6 concludes the paper by discussing potential directions for future research.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

2 Related Work

This section reviews the prior research on Sign Language Translation (SLT) and the current state of the art in Neural Machine Translation (NMT). In the field of sequence-to-sequence learning, Deep Learning (DL) [9] has become popular and gained state-of-the-art performance across a variety of domains, including Speech Recognition [10, 11], Computer Vision [12], Image Captioning [13], and more recently Continuous Sign Language Recognition and Translation (CSLRT). The sign translation protocols and their objectives, listed in Table 1, have become standard in the research community in recent years.

Many researchers of CSLRT have mainly explored handcrafted non-manual [14] and manual [15, 16] features' intermediate representations, and the temporal changes in these features [17, 18]. Since the inception of Deep Learning, CSLRT researchers have actively accepted Convolutional Neural Networks (CNN) [19, 20] for spatial feature extraction, and recurrent neural networks (RNN) [3, 21], bidirectional RNN [22], Long Short Term Memory (LSTM) [23], Bidirectional LSTM (BiLSTM) [21], dynamic hierarchical bidirectional GRU (DH-BiGRU) [20] for temporal modeling. Few studies can also be found on Sign Language Generation (SLG) which is continuous non-manual video generation from text [24–26]. Watkins et al. [4] investigated the mapping between sign generation and sign comprehension by altering hand dominance in a picture-to-sign matching task done by left-handed signers and right-handed signers. According to Zhou et al. [27], Gloss-to-text translation requires an extra decoding step. So, introducing multi-cue sources like hand, face, and pose in this step brings notable improvement in SL translation performance. The authors claim a BLEU-4 score of 23.65 combining CNNs for spatial feature extraction and RNNs, for temporal feature modeling. Zuo et al. [28] proposed a spatial attention module as a part of the visual module and guided it by pre-extracted pose key points, which can force the visual module to focus on informative regions, leading to spatial attention consistency.

Aiming et al. [35] proposed a Self-Mutual Knowledge Distillation (SMKD) method, which forces the visual and contextual modules to focus on short-term and long-term information and enhances the discriminating power of both the visual module and contextual module simultaneously. Yutong et al. [31] proposed a framework named TwoStream-SLR which adopts two streams to model RGB videos using CNN and keypoint sequences for sign language recognition using LSTM. Lianyu et al. [33] observed that CSLR methods, usually process frames independently, and thus fail to capture cross-frame trajectories for sign identification. The authors proposed a correlation network (CorrNet) to explicitly capture and leverage body trajectories across frames to identify signs. In their work, the identification module dynamically highlights body trajectories within correlation maps, allowing it to capture and understand local temporal movements effectively. As a result, the features produced by this module provide a comprehensive understanding of temporal movements, which is essential for accurately recognizing a sign.

The formalization of neural sign language translation within the NMT framework for both end-to-end and pre-trained environments was done by Camgoz et al. [2] in 2018. It enables us to collaboratively learn spatial representations, underlying linguistic models, and sign and spoken language mapping. The authors implemented two vision-based sign language tasks, in which they divided the Sign-Gloss-Text translation task into two steps, SLR and SLT. They also introduced the RWTH-PHOENIX-Weather 2014T (PHOENIX14T) dataset. Their approach significantly improved the translation performance. However, it also inflicted a mid-level gloss representation information bottleneck, as Glosses are incomplete multi-channel visual sign representations in text form. So in their further work,

Table 1 Sign Language Recognition and Translation Protocols

Sign translation protocol	Protocol objective
Sign2Text	Directly translate from continuous sign videos to sentences in spoken languages without using any intermediary representation, like glosses.
Sign2Gloss	Translate from ground truth continuous sign videos to sign gloss sequences.
Gloss2Text	Translate spoken language sentences from ground truth sign gloss sequences.
Sign2Gloss2Text	Employ CSLR models to convert sign language videos into gloss sequences, which are later utilized to resolve text-to-text issues by training a Gloss2Text network with the CSLR estimation.
Sign2Gloss→Gloss2Text	To extract gloss sequences from sign language videos, use CSLR models. Instead of commencing to train text-to-text translation networks from scratch, employ the most effective Gloss2Text network, which has been trained with ground truth gloss annotations.
Sign2(Gloss + Text)	Continuous recognition and translation of sign language are learned jointly.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Camgoz et al. [36] proposed a sign language transformer to use gloss supervision without limiting the learned representations. Using gloss annotations, they trained the transformer encoders to recognize glosses from sign videos. Then, they pass the recognized gloss along with the learned representation to the transformer decoders, trained to generate spoken language sentences one word at a time using cross-attention. Although this joint strategy simultaneously resolves two interrelated sequence-to-sequence learning issues and improves performance significantly, it left scope for the researchers to improve its performance by considering a few issues including the relative context of the input tokens for increased accuracy, optimizing the huge memory and computational needs of transformers, etc. A few works in the literature explore various relative context-preserving techniques for the sign language transformer. Ning et al. [8] proposed a Polar Relative Positional Encoding mechanism for explicitly encoding relative positional relations regarding direction and range. Relative positional relations modeling enables their model to extract the spatial relations between objects on the frames and easily exploit spatial relations described in natural language. The Gated Recurrent Unit-Relative Sign Transformer (GRU-RST) [5] is an approach that suggests learning CSLRT jointly by encoding relative positions. Incorporating relative position during the Multi-Head Attention, GRU (bidirectional RNN) serves as the Transformer's relative position encoder. Similarly, Liutkus et al. [6] proposed Stochastic Positional Encoding, and Yue et al. [7] proposed using a positional matrix for encoding the relative positions of frames.

Few researchers explored, utilizing multimodal data to sign language transformers. Yin et al. [37] proposed a translation system based on Spatial-Temporal Multi-Cue (STMC) transformer and tested their work on PHOENIX14T corpora. Using Transformer on PHOENIX14T dataset their model attained a BLEU-4 score of 24.90. Jiang et al. [29] introduced Skeletor, a transformer model that learns to represent the 3D human skeleton posture as a spatiotemporal manifold. Skeletor is a strong precondition over-body posture embedded in the transformer network and is trained for correcting noisy 3D skeletal postures utilizing implicit temporal continuity. In their further work, Jiang et al. [38] introduced SAMSLR, a skeleton-aware multi-modal SLR framework, which uses multi-modal information to increase sign recognition rates. SAMSLR improves sign recognition accuracy, especially with low video quality. Hezhen et al. [32] proposed the self-supervised pre-trainable SignBERT+ framework with model-aware hand prior incorporation. Simultaneously, researchers are making significant efforts to address data scarcity issues. Zhou et al. [39] suggest creating an inverse SLT path and using it to augment parallel training data. This approach is called a sign-back-translation which involves generating synthetic parallel data. Chen et al. [30] propose a multi-modal pretraining approach that utilizes transfer learning across different modalities for sign language translation. The authors propose progressive pretraining, in which the translation model is first pretrained on a particular corpus, after which a second within-domain pretraining on the Gloss2Text task is performed. The authors claim a BLEU 4 score of 28.39. The contemporary advancements in Sign Language Generation and Sign Language Translation can be found in Rastgoo et al. [40], and Adrián et al. [41] respectively. Recently Zuo et al. [34] introduced a novel approach for real-time CSLR to overcome limitations in offline CSLR, where processing the entire video sequence leads to high latency and memory use. The authors address these limitations by implementing an online CSLR system that segments continuous sign videos into isolated sign components and processes them in real time through a sliding window strategy. Table 2 summarizes the literature on transformers, their targeted applications, and Position Embedding (PE) techniques proposed if any.

Several studies highlight the potential of edge computing to support complex, real-time AI applications on resource-constrained devices, enhancing responsiveness, privacy, and energy efficiency across various fields. Chen et al. [42] review how deep learning can be integrated with edge computing to reduce latency, improve privacy, and enable adaptable real-time applications. They address the challenges posed by limited edge resources and propose solutions such as model compression and distributed processing, with applications in areas like autonomous vehicles and healthcare. Di Nardo et al. [43] focus on emotion recognition using low-power, AI-optimized edge architectures, emphasizing the role of specialized hardware and model optimizations that enable real-time emotion recognition in privacy-sensitive domains such as healthcare and human-computer interaction. Mattia et al. [44] examine the use of edge computing to enhance real-time image processing on mobile devices. By offloading intensive tasks to edge servers, mobile devices benefit from reduced latency and greater efficiency, supporting applications such as augmented reality and mobile gaming.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Comparison of works from the literature on transformer and positional encoding-based techniques

Reference	Year	Application	Transformer-based/Positional Embedding-based Techniques
Yin et al. [14]	2020	Sign translation	1. a translation system based on STMC transformer models for tokenization 2. features are analyzed using a BiLSTM, followed by a two-layer translation transformer
Neena et al. [5]	2021	Sign translation	1. a bidirectional RNN, GRU, serves as the Transformer model's relative position encoder
Liutkus et al. [6]	2021	Music generation	1. use Stochastic Positional Encoding 2. achieve PE in the keys domain, without computing attention matrices
Yue et al. [7]	2021	Speech recognition	1. use a positional matrix for relative position vectors 2. transform the key vectors based on positions instead of adding positional vectors to them in the SA layer
Ning et al. [8]	2021	Video-language segmentation	1. propose a Polar RPE mechanism for encoding relative positions in terms of direction and range 2. RPE enables the model to extract the spatial relations between objects on the frames
Jiang et al. [29]	2021	Body pose estimation, sign translation	1. use sine and cosine functions of different frequencies to encode the 3D skeleton positions 2. introduce Skeletor, a body pose estimator transformer model, trained to correct noisy skeletal postures
Zhou et al. [27]	2022	Sign translation	1. use the BiLSTM as an encoder to process long-term dependencies 2. propose a segmented attention mechanism (SA-LSTM) autoregressive decoder for multi-cues
Chen et al. [30]	2022	Sign transllation	1. use a multi-modal pretraining approach to cope with the data scarcity issue 2. progressive pertaining
Zuo et al. [28]	2022	Sign translation	1. integrate a spatial attention module into the visual module to guide it toward informative regions 2. use spatial attention consistency (SAC) constraint to focus on informative regions
Yutong et al. [31]	2022	Sign translation	1. propose a TwoStream-SLR that adopts two streams to model RGB videos and keypoint sequences 2. use Sign Pyramid Network (SPN) to fuse the features
Hezhen et al. [32]	2023	Sign translation	1. propose SignBERT+, the first model-aware pre-trainable framework 2. self-supervised pretraining on a large sign pose dataset 3. task-specific prediction heads added on top of the SignBERT+ encoder
Lianyu et al. [33]	2023	Sign translation	1. propose a CorrNet, to capture body trajectories across frames to identify signs
Zuo et al. [34]	2024	Sign translation	1. use a pre-trained TwoStream-SLR model to segment continuous sign videos into isolated signs 2. a sliding window approach processes sign video clips in real-time for online recognition 3. reduce latency and memory use

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3 Theoretical background

This section illustrates the role of the position encoding and attention mechanism in transformers. This is needed to better understand the proposed optimizations with respect to CSLRT.

3.1 The transformer

Transformer architecture was first recommended in language translation by Vaswani et al. [45], for English-to-German and English-to-French. It has a substantially higher level of parallelization since it uses an attention-based encoder-decoder architecture. The attention mechanism adapted by the Transformer architecture is known as Scaled Dot-Product Attention e_{ij} as given in Eq. 1,

$$\mathrm{e}_{ij} = \frac{(\mathrm{x}_i \mathrm{W}^{\mathrm{Q}})(\mathrm{x}_j \mathrm{W}^{\mathrm{K}})^T}{\sqrt{\mathrm{d}_z}} \tag{1}$$

where x_i, x_j are the vector representations of input words with dimension d_z for which the similarity score is computed. Q, and K, are query, and key vectors, respectively. W^Q, and W^K are the learnable parameter matrices (unique per layer and attention head) for query and key, respectively. Each output element, z_i is calculated as a weighted summation of linearly transformed input as given in Eq. 2. W^V is the learnable weight matrix for value vector V during training [45] which is unique per layer per head.

$$\mathrm{z}_i = \sum_{j=1}^{n} \alpha_{ij} (\mathrm{x}_j \mathrm{W}^{\mathrm{V}}) \tag{2}$$

Here, α_{ij} is a softmax function to obtain similarity value as attention. In practice, the attention function on a set of queries simultaneously packed together into a matrix Q is computed as given in Eq. 3. The keys and values are also packed into matrices K and V respectively. The dot product QK^T is called the Attention Filter.

$$\text{Attention}(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{Dh}}\right)V \tag{3}$$

In addition, instead of performing single attention, the Transformer uses multi-headed attention (MHA) to capture information from numerous representation subspaces at various places as given in Eq. 4. Here $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$ and W^O is learnable output weight matrix.

$$MHA(Q, K, V) = Concat\left(head_1, ., head_h\right)W^O \tag{4}$$

3.2 Positional Encoding

One of the many reasons transformers have proven immensely useful than RNNs is that they work on inputs in parallel. RNNs' recurrent structure prevents them from accounting for long-distance relationships, whereas transformers do not face this problem as they can see the entire sequence. This implies, however, that transformers are not proficient enough to learn frames' sequence order and need positional encoding to convey the model where a particular token is present within the context of a complete sequence. Otherwise, transformers would treat the following statements 1 and 2 similarly.

Statement 1: "The friendliest place is by the sea"

Statement 2: "The place is by the friendliest sea"

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

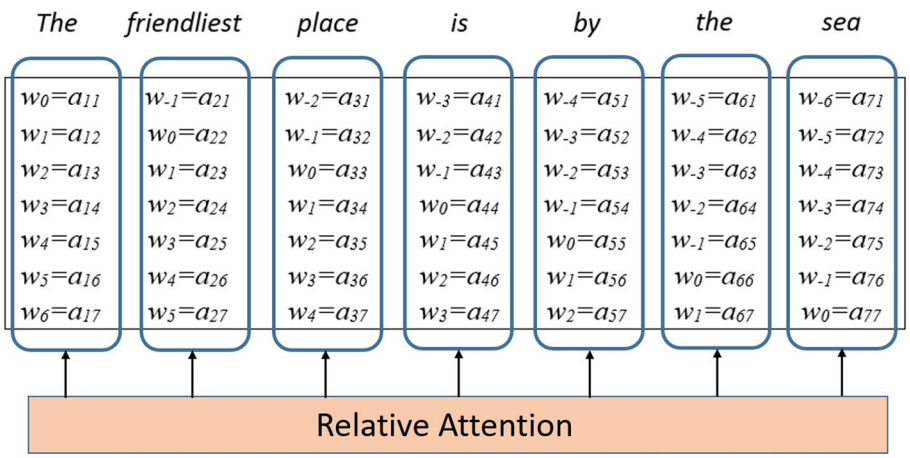


Fig. 2 Positional relationship pairwise notation

Statements 1 and 2 use the same words in a different order, which significantly changes their meanings. In Statement 1, it suggests that a place by the sea is welcoming, while in Statement 2, it implies that the sea itself is friendly Therefore, as introduced by Vaswani et al. [45], learned positional encoding is used to convey the absolute positions of all tokens (Eqs. 5 and 6). They encode the absolute order of tokens so that the transformer is aware of the sequential nature of data.

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \tag{5}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \tag{6}$$

Here *i* is the dimension and *pos* is the position in the sequence. The positional encoding has a sinusoid for each dimension. Sinusoidal absolute positional encoding, as opposed to learned absolute positional embedding [5, 7, 36, 45, 46], enables extrapolation of the model to any sequence.

Absolute Position Encoding (APE) without considering the relative position of the words in sequence is of limited significance. Relative Positional Encoding (RPE) is a more advanced approach that encodes the relative distances between words rather than their absolute positions from the start of the sequence. It allows the model to focus on the relative order and distances between words, which is often more important for understanding syntax and meaning. Shaw et al. [47] introduced RPE and proposed that with relative representations, each token has as many positional encodings as tokens in the sequence to describe the relationship between them. For instance, Table 3 shows positional relationships between the words for the example sequence, "The friendliest place is by the sea".

In this table, a row represents the relative positions of the surrounding words for the reference word for which RPE is computed (highlighted in bold). For example, for the word *friendliest*, positional encoding is (−1, 0, 1, 2, 3, 4, 5). It can be

Table 3 Relative positional Encoding between the tokens of sequence, "am freundlichsten wird es am meer (The friendliest place is by the sea)"

Token(Position)	The(0)	friendliest(1)	place(2)	is(3)	by(4)	the(5)	sea(6)
Reference: The	**0**	1	2	3	4	5	6
Reference: friendliest	−1	**0**	1	2	3	4	5
Reference: place	−2	−1	**0**	1	2	3	4
Reference: is	−3	−2	−1	**0**	1	2	3
Reference: by	−4	−3	−2	−1	**0**	1	2
Reference: the	−5	−4	−3	−2	−1	**0**	1
Reference: sea	−6	−5	−4	−3	−2	−1	**0**

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

observed that the positional encoding describing the relationship of one word with itself is a vector w_0. w_1, w_2, and so on when moved to the right and w_{-1} and w_{-2} and so on when moved to the left.

3.3 Relative Attention

The Transformers like the Transformer-XL benefit from relative attention since it allows them to generalize and produce patterns beyond the length they were trained on generating continuations that more precisely match the context. As shown in Fig. 2, self-attention can be modified to capture relative attention. Each token in sequence $x = (x_1, ..., x_l)$ of l elements where $x_i \in R^{d_x}$, gets a new representation sequence $z = (z_1, ..., z_l)$ (Eq. 7) of the same length where $z_i \in R^{d_z}$, which is passed further in the transformer module for Scaled Dot-Product Attention e_{ij} as shown in Eq. 8 [47]. In relative attention, the attention score incorporates relative positional embeddings. This means that the attention score between two tokens depends not completely on their content (queries and keys) but also on their relative distance. The attention scores are modified to include an extra term for relative positions (Eq. 9)

$$z_i = \sum_{j=1}^{n} \alpha_{ij}(x_j W^V + a_{ij}^V) \tag{7}$$

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}} \tag{8}$$

$$\text{Relative Attention}(Q, K, V, r) = \text{Softmax}\left(\frac{QK^\top + QR^\top}{\sqrt{d_k}}\right)V \tag{9}$$

3.4 Large vision model for sign language

Large Vision Model (LVM) processes visual data and applies the principles of the Transformer architecture to understand and translate sign language. While running a full-scale LVM on an edge device is difficult, it is possible using knowledge distillation, optimized architectures, model compression, and hardware acceleration [48, 49]. By applying these techniques, an edge device can run a scaled-down version of an LVM, making advanced vision tasks, like sign language translation, feasible. Knowledge distillation is a technique for mobile optimization, where a smaller "student" model is trained to replicate the behavior of a larger, more complex "teacher" model. The student model, being smaller and more efficient, is better suited for mobile devices still retaining much of the teacher's performance [50]. Optimized architectures like MobileNet and EfficientNet are designed for mobile efficiency, offering reduced computational overhead when handling complex tasks. Lightweight versions of the Transformer architecture, known as Transformer Lite variants, have also been developed for mobile deployment [51]. On-device inference frameworks such as TensorFlow Lite allow for the conversion, optimization, and deployment of models directly onto mobile devices. These frameworks optimize models to run efficiently with minimal resource consumption. Model compression methods like pruning, quantization, and weight sharing are commonly employed. Pruning removes less important weights or neurons, reducing the model's size without greatly affecting accuracy. Quantization lowers the precision of weights, such as converting from 32-bit to 8-bit integers, cutting down the size and computational requirements. Weight sharing further minimizes the number of unique parameters by reusing weights across different model layers.

Neubig et al. [52] introduced a powerful set of techniques called "neural sequence-to-sequence models". They proposed that instead of processing different examples separately, mini batching with batch normalization using a few tricks

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

to complete the simultaneous processing of *n* training examples and grouping similar operations significantly improves computational performance. This is possible because modern hardware (particularly GPU) has efficient vector processing instructions that can be exploited with appropriately structured inputs. Batch normalization is a method used in deep learning to enhance training stability, speed, and overall performance by normalizing each layer's inputs. It standardizes data distributions by computing the mean and variance within each mini-batch to ensure consistent data distributions. This technique is widely adopted in various neural network architectures, particularly deep networks, due to its benefits for efficient training and model accuracy. Additionally, several techniques have been used to reduce the computational load while maintaining performance when running LVMs on mobile devices.

4 Proposed work

4.1 Proposed method

This work introduces the SignEdgeLVM, an enhanced variant of the Sign Language Transformer (SLTr) [36] for edge devices. The key innovation of the SignEdgeLVM lies in its use of a Global Relative Attention Matrix (GRAM) and Dynamic Point Frame Sampling (DPFS) module. The GRAM captures arbitrary relationships between the input frames in the sign language videos, enhancing the model's capacity to understand temporal dependencies and optimize memory and computational needs without compromising model performance. In addition to the GRAM optimization, frame sampling within the knowledge distillation context has been used to further streamline the model. During training, the teacher model processes all frames from the video input, while the student model (SignEdgeLVM) uses a sampled frames subset. This approach significantly reduces the data volume that the SignEdgeLVM needs to process. By training the SignEdgeLVM on the outputs of the teacher, SignEdgeLVM learns to perform effectively despite handling fewer frames, which increases its efficiency without sacrificing accuracy. Frame sampling therefore plays a critical role in reducing the computational complexity of the SignEdgeLVM model.

Ablation Study: The proposed work experimented with three updated versions of SLTr [53] to assess the proposed optimizations. Following are the versions:

1. *Proposed model with GRAM* This version integrates the relative positional information for the encoder with memory optimization, applying the GRAM mechanism.
2. *Proposed model with DPFS*This version incorporates knowledge distillation using DPFS, where the student model handled a subset of video frames, improving efficiency.
3. *SignEdgeLVM*This version combines the GRAM and DPFS strategies.

Our experiments demonstrate that the SignEdgeLVM is highly efficient in terms of both memory usage and processing speed, making it ideal for edge device applications. Figure 3 shows the overview of the proposed SignEdgeLVM.

4.1.1 Sign language recognition and translation transformers

The Sign Language Recognition and Translation process begins by embedding sign language videos (SE) and spoken language words (WE) as the source and the target tokens respectively as shown in Fig. 3. It projects a one-hot-vector representation of the words into a denser space using a linear layer that is initialized from scratch during training to get an intermediate representation for words sequence $(m_u..m_U)$ from the sequence of embedded spoken language words $(W_u..W_U)$. Batch-normalized pretrained CNNs followed by ReLU have been used as spatial embeddings to embed video tokens [2, 46] to get an intermediate representation for frames$(f_t..f_T)$ from the sequence of embedded sign language video frames $(I_t..I_T)$. At this stage RPE is added to $(m_u..m_U)$ and $(f_t..f_T)$ to get relative position encoded words and frames as $(\hat{m}_u..\hat{m}_U)$ and $(\hat{f}_t..\hat{f}_T)$, which are further given as inputs to Relative Attention Layers of Sign Language

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

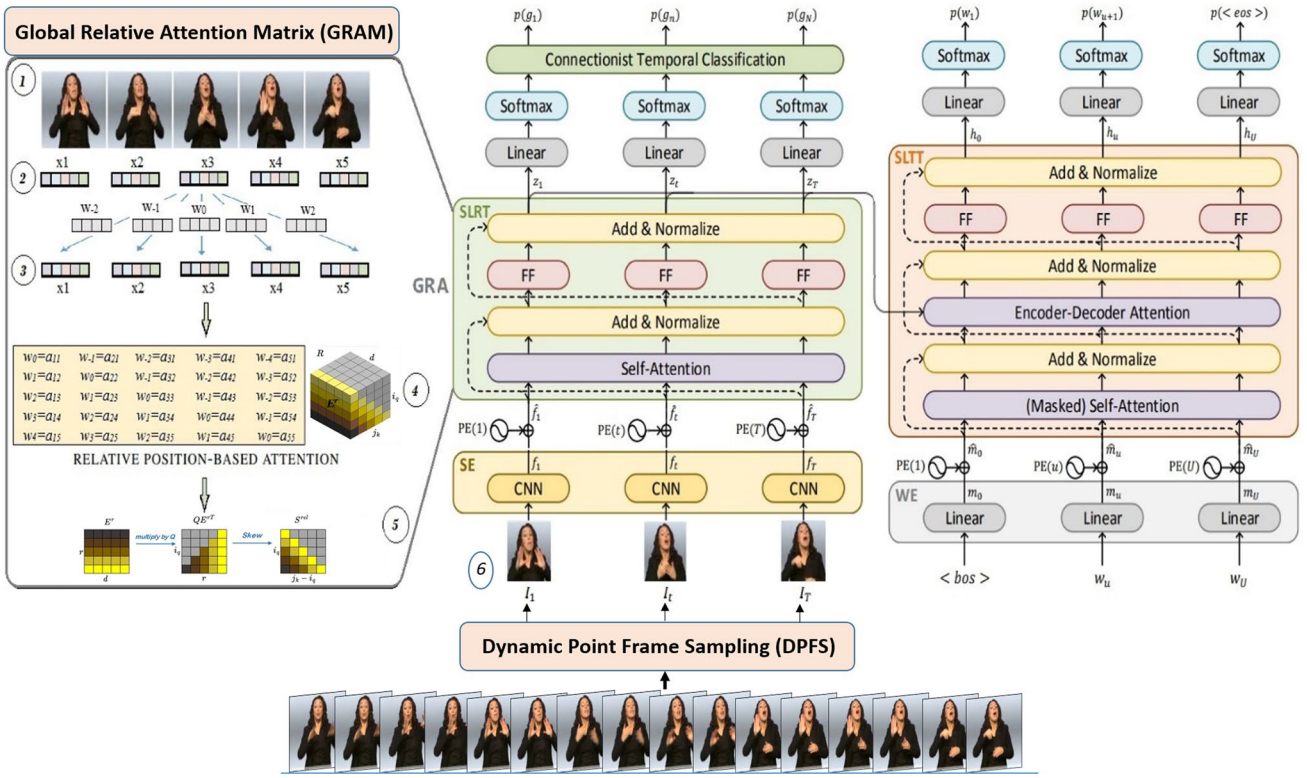


Fig. 3 A detailed overview of the SignEdgeLVM model. 1. Input Sequence, 2. Input Embedding, 3. Relative Positional Encoding, 4. GRAM and Global Relative Attention, 5. Skewing Operation, 6. Sampled Frames

Translation Transformer (SLTT) and Sign Language Recognition Transformer (SLRT) respectively. The inputs to SLRT are first modeled by a Self-Attention layer which learns the contextual relationship between the frame representations. Further, a non-linear pointwise Feed Forward layer (FF) is applied to the output of the Self-attention layer. Residual connections and normalization are applied after each operation to aid training. The spatiotemporal representation of the frame is denoted by the notation $z_t = SLRT(f_t|f_{1:T})$. At time step t, SLRT generates it from the spatial representations of every video frame, $f_{1:T}$. We introduce intermediate supervision to help our networks learn an accurate sign representation that will help with the main task of translation. A linear projection layer followed by a softmax activation has been used to extract frame-level gloss probabilities, $p(g_t|V)$, given spatio-temporal representations, $z_{1:T}$. Then, by marginalizing over all potential videos V to glosses G alignments, Connectionist temporal classification (CTC) has been used to get $p(G|V)$. The CSLR loss, L_R is then calculated using the $p(G|V)$.

The SLTT, an autoregressive transformer decoder model, takes advantage of the spatio-temporal representations learned by the SLRT. The process starts by adding the unique sentence-beginning marker, $< bos >$ to the target spoken language sentence S. The spatiotemporal representations generated by the SLRT are used in the SLTT. The positional encoded word embeddings are then extracted. These embeddings are provided to a masked self-attention layer. This masking is essential since the SLTT will not have access to the output tokens that follow the reference token decoded at the time of inference. An encoder-decoder attention module, which learns the mapping between source and target sequences, receives the combined representations from the SLRT and SLTT self-attention layers. Further, a non-linear point-wise Feed Forward layer receives the outputs of the encoder-decoder attention. Similar to the SLRT, residual connections and normalization follow each operation. The SLTT learns to generate one word at a time until it produces the special end-of-sentence token, $< eos >$. It is trained by decomposing the ordered conditional probabilities that make up the sequence-level conditional probability $p(S|V)$, which are then used to compute each word's cross-entropy loss L_T. To jointly train the networks, joint loss term L is minimized, which is

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

a weighted sum of the recognition loss L_R and the translation loss L_T as given in Eq. 10, where during training significance of each loss function is decided by the hyperparameters $\lambda_R = 5.0$ and $\lambda_T = 1.0$.

$$L = \lambda_R L_R + \lambda_T L_T \tag{10}$$

A greedy search is performed during the training and development stages to decode both spoken language sentences and gloss sequences. At inference time, beam search decoding is performed [52], with widths ranging from 0 to 10.

4.1.2 Global relative attention matrix (GRAM) and relative attention

The proposed work applies the GRAM optimization to sign translation and highlights the drawbacks of the quadratic memory footprint of the solution proposed by Shaw et al. [47]. In machine learning and neural networks, memory footprint refers to the memory required to store model weights and the intermediate outputs or activations produced at each network layer during processing. The implementation suggested by Shaw et al. [47] instantiates an intermediary tensor R of dimension (L, L, Dh) for each head, containing the embedding corresponding to the relative distances between all keys k and queries q. Here, L is the sequence's length. Query matrix Q is then transformed into an $(L, 1, Dh)$ tensor so that it can be multiplied with R^T to get $S^{rel} = QR^T$, where dimension $D_h = D/H$. Here, D denotes the hidden state dimension and H denotes the number of heads. For each head that interacts with queries, the relative embeddings are learned independently and generate output S^{rel}, an $L \times L$ dimensional logits matrix which modifies the attention probabilities for the individual head as given in Eq. 11, where $S^{rel} = QR^T$ (path 1 in Fig. 4).

Overall this implementation results in a total space complexity of $O(L^2D)$ for each layer of the transformer having H heads. This huge memory bottleneck can be improved to $O(LD)$ by using a clever skewing operation (path 2 in Fig. 4) [54]. Learning relative position representation in a sequence includes learning a different relative position embedding tensor E^r of dimension (H, L, D_h), which has an embedding for each permissible pairwise distance $r = j_k - i_q$, between a key k and query q in position j_k and i_q respectively. On multiplication of Q by E^r, the relative position embedding terms required from QR^T (Eq. 9) are already present in the resultant matrix QE^{rT}. QE^{rT} at (i_q, r) entry contains the dot product of the query q at index i_q with the embedding of relative distance r.

$$\text{Relative Attention}(Q, K, V) = Softmax\left(\frac{QK^T + S^{rel}}{\sqrt{D_h}}\right)V \tag{11}$$

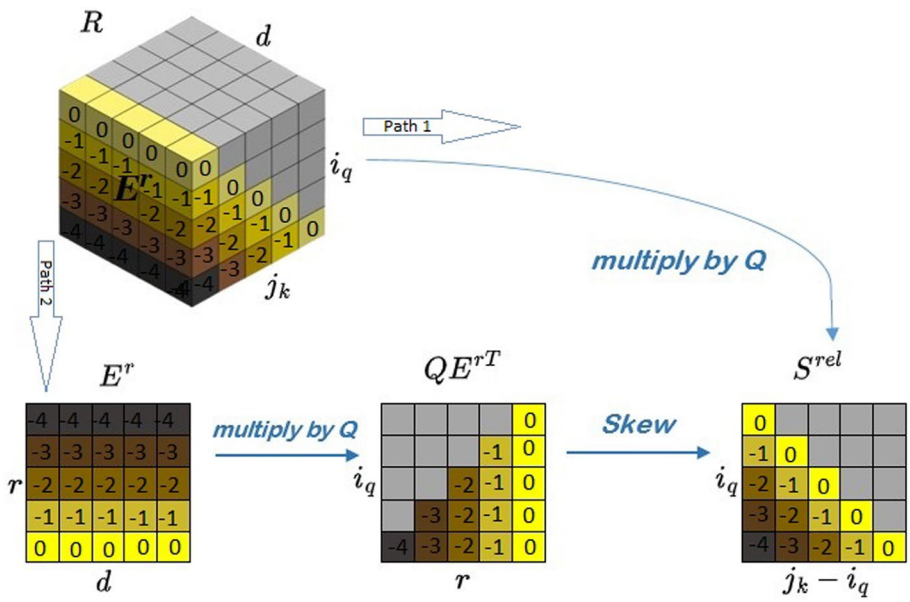


Fig. 4 Skewing Operation: Path 1 represents obtaining S^{rel} directly (without Memory Optimization), Path 2 represents obtaining S^{rel} using Skewing Operation (with Memory Optimization)) [54], For better understanding instead of logits, relative positions are shown

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

However, to match up with the indexing in QK^T, as given in Eq. 11, each relative logit (i_q, j_k) in the matrix S^{rel} should be the dot product of the query in position i_q and the embedding of the relative distance $j_k - i_q$. To relocate the relative logits to their correct places, the skewing operation is applied to QE^{rT}. Given below is the outline of the steps in the Skewing Algorithm [54]. It takes QE^{rT} as input and provides S^{rel} as output:

- *Step 1*: Pad a dummy column vector of length L before the leftmost column.
- *Step 2*: Reshape the matrix to have shape $(L + 1, L)$.
- *Step 3*: Slice that matrix to retain only the last l rows and all the columns, resulting in a (L, L) matrix again, but now absolute-by-absolute indexed, which is the S^{rel}.

Relative Positional Embedding can be shared across multiple attention heads in a layer of the Transformer. As a result, optimal memory usage is achieved when using a single E^r matrix to handle all heads rather than producing numerous ones. The reader is strongly encouraged to refer to skewing operations [54] for more details as we have presented the idea concisely here for brevity.

4.1.3 Dynamic point frame sampling (DPFS)

Frame sampling is a technique used in video processing where specific frames are selected or sampled from a video sequence, rather than processing every single frame. This method is commonly employed to reduce the amount of data that needs to be processed, to focus on specific parts of a video. Frame Sampling can be done using various methods. Time-based extraction involves capturing frames at regular intervals known as Interval Frame Sampling. Scene Change Detection focuses on extracting frames when there's a significant alteration in the visual content, ensuring key moments are captured. Motion Detection targets frames based on the presence or absence of motion, allowing the selection of frames during critical actions or when the scene is static. Lastly, Quality-based Extraction ensures that only high-quality frames are chosen, filtering out blurry, poorly lit, or noisy ones. Frame Sampling method helps reduce computational load, handle long video sequences more efficiently, and focus on the most relevant frames for the task. Towards this end, we utilize a Siamese CNN architecture (SNN) [55, 56] with two pathways that share parameters where each pathway follows GoogLeNet architecture. To learn the parameters, the loss function as Piecewise Ranking (PR) loss [55] is adopted as it introduces relaxation of the ground truth score and makes the network more stable for a subjective task. Supposing the input pair of frames fed into the network is (I_1, I_2), and $P(I_i)$ is its estimated value from the SNN, $Dp = P(I_1) - P(I_2)$ is the predicted value difference.

4.1.4 Sampling contiguous frames with maximum difference

The sampling of contiguous frames with maximum difference strategy focuses on selecting frames that are visually distinct from each other. Visually distinct frames have large differences in structural features, or content, such as a sudden movement or scene change. The proposed work introduces a feature-based (feature embeddings from a neural network) Dynamic Point Frame Sampling (DPFS) algorithm that employs sampling contiguous frames with the maximum difference algorithm. Algorithm 1 presents the steps for selecting visually distinct frames based on feature-based maximum differences. In Step 1, the algorithm extracts feature embeddings for each video frame using a pretrained feature extractor model, a CNN. In Step 2, the first frame is initialized into the selected set S, and its embedding is set as the reference for comparison. In Step 3, the algorithm iteratively selects the frame with the maximum feature-based difference from the last selected frame. In Step 4, the algorithm returns the sorted set of k selected visually distinct frames.

By segmenting the video and later applying frame sampling, a balance between reducing redundancy and preserving temporal information can be set. This approach helps maintain local context within segments while enabling more efficient processing. In our set of experiments, we demonstrate the effect of segment sizes on accuracy (Fig. 7).

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Algorithm 1 Feature-based frame sampling with maximum difference

Require: Video frames $F = \{f_1, f_2, \ldots, f_n\}$, Feature Extractor, Number of frames to sample k
Ensure: Selected frames $S = \{s_1, s_2, \ldots, s_k\}$
Step 1: Extract Features from Each Frame
for each frame $f_i \in F$ **do**
  Extract feature embedding $\mathbf{e}_i = \text{FeatureExtractor}(f_i)$
end for
Store feature embeddings $E = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n\}$
Step 2: Initialize Selection
Initialize selected set $S = \{f_1\}$, last_selected $= \mathbf{e}_1$
Step 3: Iterative Frame Selection
for $i = 2$ to $k + 1$ **do**
  Initialize $max\_diff = -\infty$
  for each frame $f_j \in F \setminus S$ **do**
    Compute difference $\text{diff}(f_j, \text{last\_selected}) = \|\mathbf{e}_j - \mathbf{e}_{\text{last\_selected}}\|$
    if diff $> max\_diff$ **then**
      $max\_diff = \text{diff}$
      $f_{\max} = f_j$
    end if
  end for
  Add $f_{\max}$ to S
  Update last_selected $= \mathbf{e}_{\max}$
end for
Step 4: Return Selected Frames
return S in sorted order

5 Experiments and Results

5.1 Dataset

The PHOENIX14T dataset [2] is a continuous SLT corpus with an extensive vocabulary and serves as a crucial benchmark for CSLR. It has evolved from the PHOENIX14 corpus, now enhanced with a focus on translation-specific tasks. The PHOENIX14T including parallel sign language videos, gloss annotations, and corresponding human-annotated translations, is uniquely suited for jointly training and evaluating SLR and SLT algorithms. The corpus contains 1066 distinct signs from unconstrained continuous sign language by 9 distinct signers. The spoken German language translations for these videos have a vocabulary of 2887 words. There are 7096, 519, and 642 samples in training, development, and test sets respectively. The TV studio manages the lighting and signer placement in front of the camera. Each video has a resolution of 210 x 260 pixels and 25 interlaced frames per second. The TV station's broadcasting method is accountable for the inadequate spatial and temporal resolution. The PHOENIX14T dataset, with its aligned video, gloss annotations, and translations, is highly suited for translation tasks in sign language research. Its extensive size aids in developing more generalizable models, contributing to robust findings across different scenarios.

5.2 Implementation

We developed the SignEdgeLVM in the PyTorch framework [57] by partially adapting the open-source Sign Language Transformers[1] https://github.com/neccam/slt. Only CTC beam search decoding was carried out using Tensorflow [16, 58] implementation. The training has been carried out on NVIDIA Quadro M6000 12GB RAM. Transformer networks were trained from scratch, using Adam optimizer [16], and various batch sizes ranging from 16 to 128 with a learning rate of 10^{-3} ($\beta_1 = 0.9, \beta_2 = 0.998$) and a weight decay of 10^{-3}. Table 4 presents additional information on related hyperparameters.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 4 Model Configuration

Hyper-parameter	Value
Number of encoders	3
Number of decoders	3
Number of heads	8
Transformer hidden units	512
Initializer	Xavier
Optimizer	Adam
Learning rate scheduling	Plateau
Embedding dimension	512
Dropout rate	0.1
Recognition beam size	1 to 10
Translation beam size	1 to 10

The transformer model was converted into the TensorFlow Lite format (.tflite) using the TFLiteConverter tool [51]. After conversion, the.tflite model was integrated into the mobile app using TensorFlow Lite's mobile APIs. A laptop with 8GB RAM, Intel Core i7-9750 H @ 2.60 GHz processor has been used as the edge device [44].

5.3 Results

Our qualitative findings are detailed in this section. Table 5 presents a few examples of the reference spoken language text and their translations generated by the SLTr, and the SignEdgeLVM, given the sign video representations under the

Table 5 Comparison of the translations generated by the proposed SignEdgeLVM with SLTr (Sign Language Transformer) [53]

Reference Text	dahinter fliet etwas khlere luft nach deutschland aber ab dienstag steigen luftdruck und temperaturen langsam wieder und es wird freundlicher. (behind it some cooler air flows to germany but from tuesday onwards air pressure and temperatures will slowly rise again and it will be friendlier.)
SLTr	wir haben schließlich ein sonniges hoch ab dienstag den dienstag das hoch für freundliches wetter verbreitet werte in küstennähe. (we finally have a sunny high from tuesday tuesday the high for friendly weather spread values near the coast.)
SignEdgeLVM	am dienstag strömt sehr kühle luft zu uns am dienstag erreichen uns am dienstag und wieder freundlicher. (on tuesday very cool air flows to us on tuesday reach us on tuesday and again friendlier.)
Reference Text	von westen breitet sich teils schnee teils regen aus auch gefrierender regen ist in der nacht dabei und es muss mit gefhrlicher glatte gerechnet warden. (from the west partly snow and partly rain is spreading, there is also freezing rain during the night and dangerous slippery conditions must be expected.)
SLTr	von westen fällt teilweise schnee glatt werden auf örtlich ein paar tropfen bringen die hier und da wird es zunehmend in schnee verwandeln. (from the west partly falling snow getting smooth locally bring a few drops here and there it will increasingly turn into snow.)
SignEdgeLVM	im westen fällt heute nacht teilweise schnee und stellenweise etwas regen im norden kann es in der nacht auch gefährlich glatt werden. (in the west there will be snow tonight and some rain in places in the north it can also be dangerously slippery at night.)
Reference Text	erst richtung sÃden so alpenrand werden die wolken dichter und zwischen donau und alpenrand kann es auch immer noch ein bisschen schneien wird aber auch im laufe des tages weniger. (only towards the south, towards the edge of the alps, do the clouds become denser and between the danube and the edge of the alps it can still snow a bit, but it will also become less during the day.)
SLTr	im süden wird es wolkiger mit zwischen donau und und und dem östlichen alpenrand gebietsweise schneit es auch leicht. (in the south it gets cloudy between the danube and and the eastern edge of the alps in some areas it also snows lightly.)
SignEdgeLVM	im süden da wird es allerdings noch wolkiger und vor allem am alpenrand die oder schneien sonst schneit es mitunter leicht. (in the south it gets even cloudier and especially at the edge of the alps or snowing otherwise it sometimes snows lightly.)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

headings Reference Text, SLTr, and SignEdgeLVM respectively. Since the PHOENIX14T dataset has annotations in German, the English translations of the generated sentences are also provided.

We assess our recognition models using the Word Error Rate (WER), a widely used metric for evaluating CSLR performance [11]. WER is computed as given in Eq. 12,

$$WER = \frac{\#insertions + \#deletions + \#substitutions}{\#words\ in\ reference} \tag{12}$$

To evaluate the performance of the proposed model on translation precision and recall, the most used metrics Bilingual Evaluation Understudy Score (BLEU) [59] (n-grams ranging from 1 to 4) as shown in Eq. 13, and Recall-Oriented Understudy for Gisting Evaluation (ROUGE) [60] as shown in Eq. 14 respectively are utilized,

$$BLEU = BP * \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{13}$$

$$ROUGE = \frac{\#n-grams\ matching\ in\ R\ and\ H}{\#n-grams\ in\ R} \tag{14}$$

Here n stands for the n-gram orders observed for n-gram precision p_n of total length N. The n-gram precision weights are denoted by w_n. The conciseness penalty, or BP, considers the potential for too-brief high-precision translation. R and H refer to reference and hypothesis texts, respectively.

To evaluate the Translation Throughput of the proposed model, the Tokens Per Second (TPS) metric has been used. TPS is a crucial metric for assessing the processing speed of generative models, particularly in NLP and large language model (LLM) tasks. It quantifies the rate at which a model generates or processes tokens/words during inference. High TPS values indicate a faster, more efficient model, which can handle or produce a greater text volume in less time [61].

WER, BLEU, and ROUGE are computed on the train, validation, and test sets. When CSLR is the primary goal, the WER metric is reduced, whereas the BLEU score is maximized for SLT, during training. From the graphs in Fig. 5, it can be concluded that the loss with Absolute Positional Encoding (for SLTr) decreases faster (training time 706 min), although it is not as steady and low enough (training time 718 min) as the Relative Positional Encoding (using GRAM). Also, recognition loss stabilizes faster as compared to translation loss using GRAM. In Fig. 6 detailed comparison of metrics WER, BLEU 1, BLEU 2, BLEU 3, BLEU 4, and ROUGE for SLTr and the proposed model with GRAM is presented. Here, pair (Batch size-PE) represents the Batch size of processed samples and Absolute/Relative positional encoding. For example, 64-R represents Batch size=64 and R=Relative

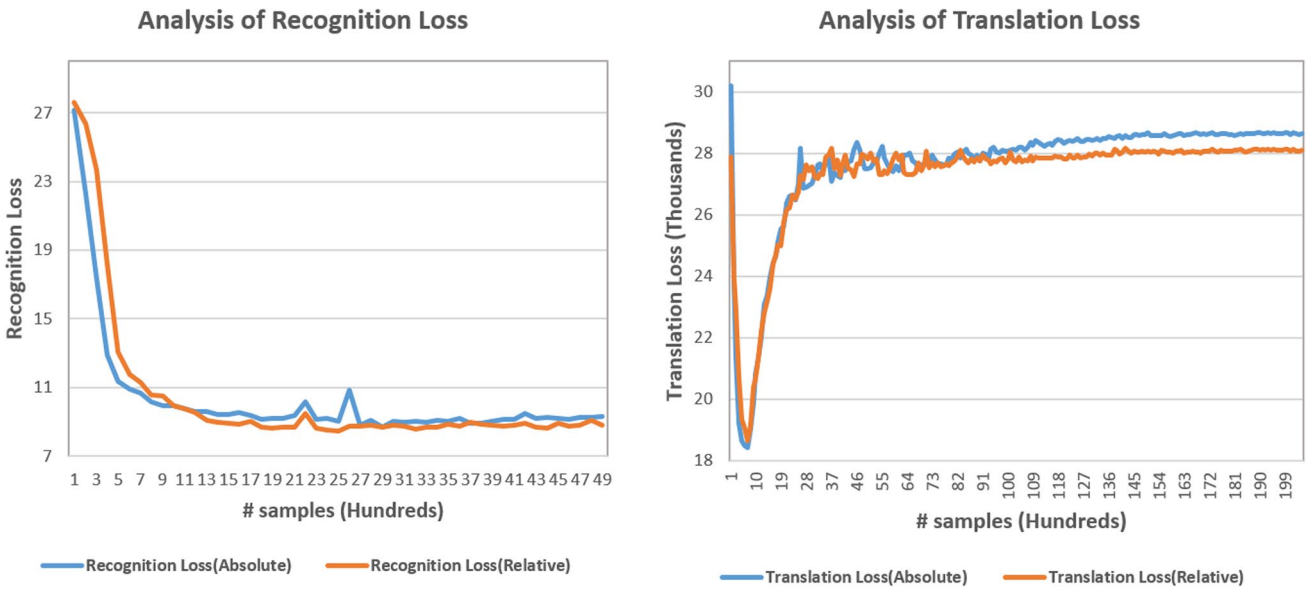
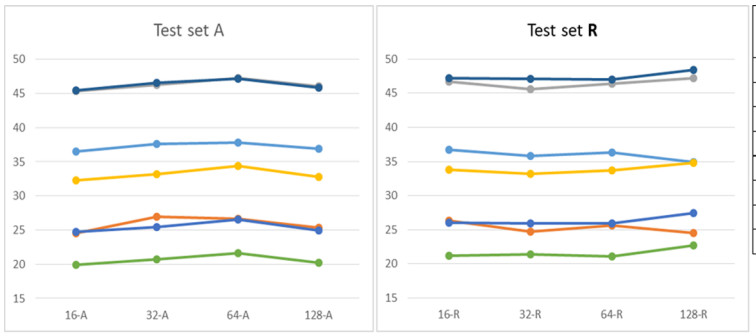


Fig. 5 Recognition Loss and Translation Loss curves during training for Absolute Positional encoding and Relative Positional encoding for batch size 128

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



Batch size -PE	Test						
	WER	DEL	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE
16-A	36.46	24.56	45.31	32.27	24.72	19.92	45.44
16-R	36.68	26.32	46.74	33.84	26.08	21.17	47.18
32-A	37.64	26.91	46.24	33.15	25.44	20.75	46.56
32-R	35.85	24.70	45.56	33.15	25.95	21.37	47.12
64-A	37.83	26.63	**47.24**	34.39	26.58	21.66	47.18
64-R	36.30	25.64	46.38	33.70	25.97	21.12	47.04
128-A	36.86	25.33	46.10	32.78	24.98	20.20	45.87
128-R	**34.87**	**24.51**	47.17	**34.82**	**27.43**	**22.68**	**48.46**

Fig. 6 Analysis of the performance and batch-size for test set: A-Absolute positional encoding used by SLTr, R-Relative positional encoding used by GRAM, the x-axis represents a batch type, and the y-axis represents metric value

positional encoding used by GRAM. A few things can be observed from these curves. WER is observed to be always reducing when RPE is considered. WER is the basic metric of comparison where the number of deletions counts for translation quality and reduces when the context of the word is preserved. BLEU measures precision i.e. how much the n-grams in the machine-generated summaries appeared in the human reference summaries; ROUGE measures recall i.e. how much the n-grams in the human reference summaries appeared in the machine-generated summaries. So, BLEU and ROUGE metric results are complementary. BLEU 1 and ROUGE metrics almost overlap when contextual information is not provided with APE (left side graphs), which means it fails to provide translation better in terms of recall. This is not the case with RPE (right side graphs) as it can be observed that ROUGE is always higher than BLEU 1 score. Also, it is observed that a larger batch size gives better translation performance. Figure 7 presents the effect of segmentation on the BLEU-4 score. 100 frames are collected from a sign video by varying the number of segments from 1 to 10 and the sampling rate from 10 to 100 respectively. It can be concluded that careful attention must be paid to segmentation strategies, and sampling rates within segments to ensure that translation accuracy is not significantly impacted.

In Table 6, we compare the proposed work with the existing transformers, using a similar line of research focusing on positioning strategies in transformers from the perspective of sign language translation. It is important to observe that ours is the only work targeting edge devices' memory and computation optimizations. As a recognized benchmark in the field, it facilitates meaningful comparisons with existing studies, helping establish standards for new model validation. Table 7 presents memory usage by the attention layers (per attention layer) with and without GRAM-DPFS. Our research considerably reduces the memory footprint using GRAM-DPFS in attention computation. We reduce the memory requirement of relative attention from L^2D to LD (78.22 MB i.e. 99.93 percent) per attention layer. This is the reduction of the memory needed by the SignEdgeLVM when compared to the SLTr. The reader is advised to note that, this reduction is specific to the results obtained by the proposed implementation with the number of heads, H=8. Figure 8 showcases the translation throughput for the SignEdgeLVM, and Teacher Model. The average translation throughput for the SignEdgeLVM, and the Teacher Model is 803 TPS, and 1935 TPS respectively.

As the SignEdgeLVM accounts for the relative positions of the words in a sequence it performs better when compared to the SLTr. This ability allows the SignEdgeLVM model to handle ambiguities and variations in sign meaning more effectively. For example, some signs carry multiple meanings depending on the context, much like homonyms in spoken languages. Without recognizing the relative positions of surrounding signs, SLTr struggles to select the appropriate translation for such ambiguous signs. However, the SignEdgeLVM model still struggles with issues such as gloss recognition due to unusual grammar structures. In continuous sign language translation, signs often blend due to coarticulation, where the movement of one sign affects the start or end of the next. The spatial arrangement and directionality of signs further influence their meaning. This makes it difficult to segment signs accurately, which can decrease model accuracy for translation.

6 Conclusion and future work

Sign language being a visual language, often takes considerable time to master. Communication barriers, such as the lack of accessible sign language resources and interpreters, hinder deaf and mute individuals' access to essential services like healthcare, courts, and public services. Transformers have demonstrated superior performance over traditional recurrent

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

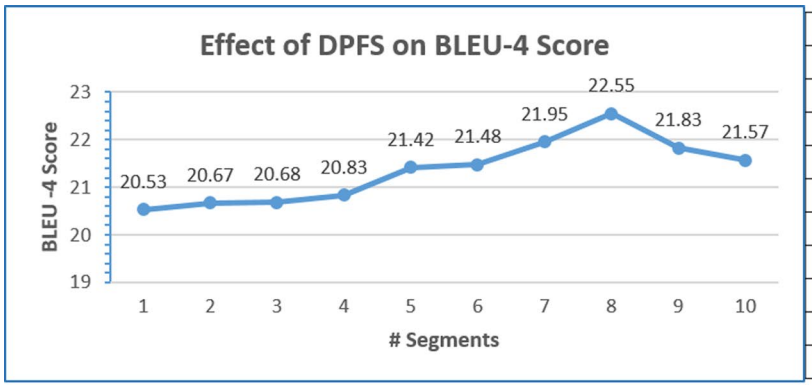
Table 6 Comparison of transformer-based approaches for sign language translation on the PHOENIX14T dataset

Reference	Year	Model	Description	BLEU-4 score	Remarks
Camgoz et al. [62]	2020	CNN+transformer encoder-decoder	Direct translation without gloss recognition	19.61	Transformer generated translations using absolute position encoding
Neena Aloysius et al. [5]	2021	CNN+GRU+ transformer with modified MHA	Direct translation without gloss recognition	19.65	Transformer generated translations with GRU encoding in place of absolute positioning and modified MHA to fetch relative positioning
Camgoz et al. [36]	2020	CNN+transformer encoder-decoder	Translation with gloss attention	21.32	Transformer generated translations using absolute position encoding
Kayo Yin et al. [14]	2020	STMC+transformer encoder-decoder	Translation with gloss attention	21.65	Transformer generated translations using absolute position encoding
Neena Aloysius et al. [5]	2021	CNN+GRU+ transformer with modified MHA	Translation with gloss attention	22.40	Transformer generated translations with GRU encoding in place of absolute positioning and modified MHA to fetch relative positioning
Zhou et al. [27]	2022	STMC + segmented attention using LSTM	Translation with gloss attention	23.65	Multi-Cue Fusion is done using Transformer Multi-Cue (TMC) blocks and Temporal Pooling (TP) layers
Chen et al. [30]	2022	pretrained model mBART	Translation with gloss attention	28.39	train mBART on the pretrained visual encoder
Hezhen, et al. [32]	2023	SignBERT+ encoder	Translation with gloss attention	25.70	Transformer generated translations using absolute position encoding with multimodal approach
SignEdgeLVM	-	CNN+ GRAM+ DPFS for Edge Devices	Translation with gloss attention	22.55	Transformer generated translations with Relative attention using GRAM, and DPFS for reduction in memory and computational complexity

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 7 Comparing the overall relative memory complexity (intermediate relative embeddings (R or E^r) + relative logits S^{rel}), for a GPU with 12GB memory assuming Hidden state Dimension D=512 (H=8, $Dh = D/H$=64), for the training input of length L

Implementation	Relative Memory	L	Memory Usage per Layer	BLEU-4 score
Without GRAM-DPFS (SLTr)	$O(L^2D + L^2)$	400	78.27 MB	20.20
Proposed model with GRAM	$O(LD + L^2)$	400	0.34 MB	22.68
Proposed model with DPFS	$O(L^2D + L^2)$	100	4.89 MB	19.72
With GRAM-DPFS (SignEdgeLVM)	$O(LD + L^2)$	100	0.05 MB	22.55



# Segments	Sampling Rate	BLEU-4 Score
1	100	20.53
2	50	20.67
3	33	20.68
4	25	20.83
5	20	21.42
6	16	21.48
7	14	21.95
8	**12**	**22.55**
9	11	21.83
10	10	21.57

Fig. 7 Effect of within segment frame sampling on BLEU-4 score for the SignEdgeLVM, #Segments is number of segments in which input video is divided, Sampling Rate is number of frames per segment

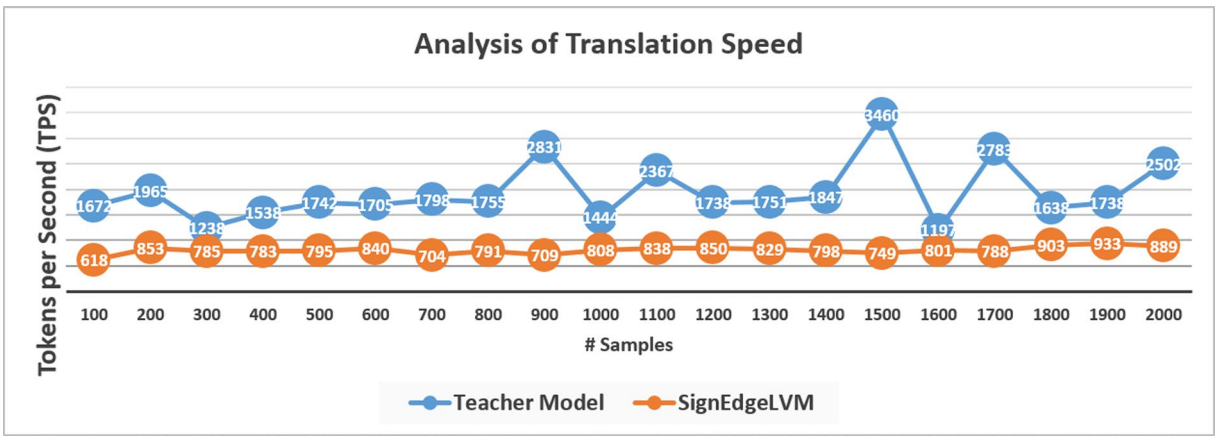


Fig. 8 Translation Throughput (Tokens per Second) [61] for the SignEdgeLVM (803 TPS), and Teacher Model (1935 TPS)

neural networks like LSTMs in SLT tasks, with faster training times, although they require more memory. In this work, we introduce a model SignEdgeLVM, that leverages GRAM and DPFS. This integration significantly boosts memory efficiency, cutting memory consumption by 78.22 MB (99.93 percent) per attention head by sharing the Global Relative Attention Matrix (GRAM) and, the Dynamic Point Frame Sampling (DPFS) algorithm that selects the most informative frames from sign video sequences, reducing the amount of data to process, thus also reducing the computational complexity of the SignEdgeLVM.

The proposed work was evaluated using the PHOENIX14T, a translation-specific enhanced dataset. It achieved a BLEU 4 score of 22.68 on the SignEdgeLVM's teacher model (with GRAM) and 22.55 on the student version of SignEdgeLVM. Additionally, a ROUGE score of 48.46 highlights the effectiveness of the SignEdgeLVM in contextual translations. From an SLT perspective, there is limited research on edge devices, leading us to claim that the SignEdgeLVM is the first model designed for lower-configured edge devices. However, overlapping signers in the training and test sets pose a limitation, making it challenging to evaluate the model's independence from specific signers. In this work, we used a

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

laptop as an edge device, achieving a computation speed of 803 TPS for SignEdgeLVM. However, we could not analyze energy consumption in this setup. In future work, we plan to implement SignEdgeLVM on dedicated embedded or edge systems, such as the Raspberry Pi or Jetson Nano/Xavier, to enable a more comprehensive evaluation, including energy efficiency [42]. At this stage, we emulated the Raspberry Pi platform using QEMU. The CPU utilization for SignEdgeLVM on a Raspberry Pi (Raspbian 4B) is approximately 68% across all cores, with RAM utilization around 200 MB during inference.

Author contributions Rina Damdoo designed the methodology, conducted the literature review and analysis, and wrote the initial draft. Praveen Kumar contributed to the revision and supervised the project.

Funding Not applicable.

Data availability The dataset analysed during the current study is available in the [RWTH-PHOENIX-Weather2014T] public repository, https://www-i6.informatik.rwth-aachen.de/~koller/RWTH-PHOENIX-2014-T/

Declarations

Ethics approval and consent to participate Not applicable.

Competing interests The authors declare that they have no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Eberhard D, Simons G, Fennig C. Ethnologue: Languages of the World, 22nd Edition, 2019.
2. Camgoz N.C, Hadfield S, Koller O, Ney H, Bowden R. Neural sign language translation. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018;7784–7793. https://doi.org/10.1109/CVPR.2018.00812
3. Koller O, Zargaran S, Ney H. Re-sign: Re-aligned end-to-end sequence modelling with deep recurrent cnn-hmms. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017;2017:3416–24.
4. Watkins F, Thompson RL. The relationship between sign production and sign comprehension: What handedness reveals. Cognition. 2017;164:144–9. https://doi.org/10.1016/j.cognition.2017.03.019.
5. Aloysius N, M G, Nedungadi P. Incorporating relative position information in transformer-based sign language recognition and translation. IEEE Access **9**, 145929–145942 (2021)
6. Liutkus A, Cífka O, Wu S, Simsekli U, Yang Y, Richard G. Relative positional encoding for transformers with linear complexity. CoRR arXiv:2105.08399 2021.
7. Yue F, Ko T. An investigation of positional encoding in transformer-based end-to-end speech recognition. In: 2021 12th International Symposium on Chinese Spoken Language Processing (ISCSLP), 2021;1–5. https://doi.org/10.1109/ISCSLP49672.2021.9362093
8. Ning K, Xie L, Wu F, Tian Q. Polar relative positional encoding for video-language segmentation. In: International Joint Conference on Artificial Intelligence 2020. https://api.semanticscholar.org/CorpusID:220484147
9. LeCun Y, Bengio Y, Hinton G. Deep learning. Nature. 2015;521:436–44. https://doi.org/10.1038/nature14539.
10. Chorowski J, Bahdanau D, Serdyuk D, Cho K, Bengio Y. Attention-based models for speech recognition. CoRR arXiv:1506.07503 2015.
11. Amodei D, Ananthanarayanan S, Anubhai R, Bai J, Battenberg E, Case C, Casper J, Catanzaro B, Chen J, Chrzanowski M, Coates A, Diamos GF, Elsen E, Engel J, Fan LJ, Fougner, C, Hannun AY, Jun B, Han TX, LeGresley P, Li X, Lin L, Narang, S, Ng A, Ozair S, Prenger RJ, Qian S, Raiman J, Satheesh S, Seetapun D, Sengupta S, Sriram A, Wang C-J, Wang Y, Wang Z, Xiao B, Xie Y, Yogatama D, Zhan J, Zhu Z. Deep speech 2 : End-to-end speech recognition in english and mandarin. In: International Conference on Machine Learning 2015. https://api.semanticscholar.org/CorpusID:11590585
12. Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. Communications of the ACM. 2012;60:84–90.
13. Khurana K, Deshpande U. Video localized caption generation framework for industrial videos. Journal of Intelligent & Fuzzy Systems, 2022;4107–4132.
14. Yin K, Read J. Better sign language translation with stmc-transformer. In: International Conference on Computational Linguistics 2020. https://api.semanticscholar.org/CorpusID:226237179
15. Jiang S, Sun B, Wang L, Bai Y, Li K, Fu YR. Sign language recognition via skeleton-aware multi-model ensemble. ArXiv arXiv:2110.06161 2021.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

16. Abadi M, Barham P, Chen J, Chen Z, Davis A, Dean J, Devin M, Ghemawat S, Irving G, Isard M, Kudlur M, Levenberg J, Monga R, Moore S, Murray D, Steiner B, Tucker P, Vasudevan V, Warden P, Zhang X. Tensorflow: A system for large-scale machine learning 2016.
17. Koller O, Forster J, Ney H. Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers. Comput Vis Image Underst. 2015;141:108–25.
18. Cooper H, Ong E-J, Pugeault N, Bowden R. Sign Language Recognition Using Sub-units, 2017;89–118. https://doi.org/10.1007/978-3-319-57021-1_3
19. Koller O, Ney H, Bowden R. Deep hand: How to train a cnn on 1 million hand images when your data is continuous and weakly labelled. 2016.https://doi.org/10.1109/CVPR.2016.412
20. Zheng J, Zhao Z, Chen M, Chen J, Wu C, Chen Y, Shi X, Tong Y. An improved sign language translation model with explainable adaptations for processing long sign sentences. Computational Intelligence and Neuroscience 2020;2020.
21. Camgöz NC, Hadfield S, Koller O, Bowden R. Subunets: End-to-end hand shape and continuous sign language recognition. IEEE International Conference on Computer Vision (ICCV). 2017;2017:3075–84.
22. Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. In: Proceedings of the International Conference on Learning Representations (ICLR), vol. abs/1409.0473 2015.
23. Amin M, Hefny H, Mohammed A. Sign language gloss translation using deep learning models. International Journal of Advanced Computer Science and Applications 2021;12(11). https://doi.org/10.14569/IJACSA.2021.0121178
24. Stoll S, Camgoz NC, Hadfield S, Bowden R. Text2sign: Towards sign language production using neural machine translation and generative adversarial networks. International Journal of Computer Vision. 2020;128:891–908.
25. Bragg D, Koller O, Bellard M, Berke L, Boudreault P, Braffort A, Caselli NK, Huenerfauth M, Kacorri H, Verhoef T, Vogler C, Morris MR. Sign language recognition, generation, and translation: An interdisciplinary perspective. Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility 2019.
26. Saunders B, Camgöz NC, Bowden R. Progressive transformers for end-to-end sign language production. ArXiv arXiv:2004.14874 2020.
27. Zhou H, Zhou W, Zhou Y, Li H. Spatial-temporal multi-cue network for continuous sign language recognition. CoRR arXiv:2002.03187 2020. arXiv:2002.03187
28. Zuo R, Mak B. C2slr: Consistency-enhanced continuous sign language recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022;5131–5140.
29. Jiang T, Camgoz NC, Bowden R. Skeletor: Skeletal transformers for robust body-pose estimation. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). 2021;2021:3389–97.
30. Chen Y, Wei F, Sun X, Wu Z, Lin S. A simple multi-modality transfer learning baseline for sign language translation. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022;2022:5110–20.
31. Chen Y, et al. Two-stream network for sign language recognition and translation. Advances in Neural Information Processing Systems. 2022;35:17043–56.
32. Hu H, Zhao W, Zhou W, Li H. Signbert+: Hand-model-aware self-supervised pre-training for sign language understanding. IEEE Transactions on Pattern Analysis & Machine Intelligence. 2023;45(09):11221–39. https://doi.org/10.1109/TPAMI.2023.3269220.
33. Hu L, Gao L, Liu Z, Feng W. Continuous sign language recognition with correlation network. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023;2529–2539.
34. Zuo R, Wei F, Mak B. Towards Online Continuous Sign Language Recognition and Translation 2024. arXiv:2401.05336
35. Hao A, Min Y, Chen X. Self-mutual distillation learning for continuous sign language recognition. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), 2021;11283–11292. https://doi.org/10.1109/ICCV48922.2021.01111
36. Camgoz NC, Koller O, Hadfield S, Bowden R. Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation 2020. arXiv:2003.13830
37. Yin K, Read J. Attention is all you sign: Sign language translation with transformers. 2020. https://api.semanticscholar.org/CorpusID:231609254
38. Jiang S, Sun B, Wang L, Bai Y, Li K, Fu YR. Skeleton aware multi-modal sign language recognition. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). 2021;2021:3408–18.
39. Zhou H, Zhou W-g, Qi W, Pu J, Li H. Improving sign language translation with monolingual data by sign back-translation. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021;1316–1325.
40. Rastgoo R, Kiani K, Escalera S, Sabokrou M. Sign language production: A review. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). 2021;2021:3446–56.
41. Núñez-Marcos A, Perez-de-Viñaspre O, Labaka G. A survey on sign language machine translation. Expert Systems with Applications. 2023;213: 118993. https://doi.org/10.1016/j.eswa.2022.118993.
42. Chen J, Ran X. Deep learning with edge computing: A review. Proceedings of the IEEE. 2019;107(8):1655–74. https://doi.org/10.1109/JPROC.2019.2921977.
43. Di Nardo E, Santopietro V, Petrosino A. Emotion recognition at the edge with ai specific low power architectures. Microprocessors and Microsystems. 2021;85: 104299. https://doi.org/10.1016/j.micpro.2021.104299.
44. Proietti Mattia G, Beraldi R. A study on real-time image processing applications with edge computing support for mobile devices. In: 2021 IEEE/ACM 25th International Symposium on Distributed Simulation and Real Time Applications (DS-RT), 2021;1–7. https://doi.org/10.1109/DS-RT52167.2021.9576139
45. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A.N, Kaiser L, Polosukhin I. Attention is all you need. CoRR arXiv:1706.03762 2017.
46. Koller O, Camgoz NC, Ney H, Bowden R. Weakly supervised learning with multi-stream cnn-lstm-hmms to discover sequential parallelism in sign language videos. IEEE TranLiutkusetsactions on Pattern Analysis and Machine Intelligence. 2020;42:2306–20.
47. Shaw P, Uszkoreit J, Vaswani A. Self-attention with relative position representations. In: Walker, M, Ji H, Stent A. (eds.) Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers), Association for Computational Linguistics, New Orleans, Louisiana 2018;464–468. https://doi.org/10.18653/v1/N18-2074 . https://aclanthology.org/N18-2074

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

48. Tabani H, Balasubramaniam A, Marzban S, Arani E, Zonooz B. Improving the efficiency of transformers for resource-constrained devices. In: 2021 24th Euromicro Conference on Digital System Design (DSD), IEEE Computer Society, Los Alamitos, CA, USA 2021;449–456. https://doi.org/10.1109/DSD53832.2021.00074 . https://doi.ieeecomputersociety.org/10.1109/DSD53832.2021.00074
49. Sanh V, Debut L, Chaumond J, Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter 2020. arXiv:1910.01108
50. Computer Vision and Image Processing. Lecture Notes in Computer Science, vol. 2010. Springer 2024.
51. Wisultschew C, Otero A, Portilla J, Torre E. Artificial vision on edge iot devices: A practical case for 3d data classification. In: 2019 XXXIV Conference on Design of Circuits and Integrated Systems (DCIS), 2019;1–7. https://doi.org/10.1109/DCIS201949030.2019.8959857
52. Neubig G. Neural machine translation and sequence-to-sequence models: A tutorial. CoRR arXiv:1703.01619 2017.
53. Kreutzer J, Bastings J, Riezler S. Joey nmt: A minimalist nmt toolkit for novices 2019.
54. Huang CA, Vaswani A, Uszkoreit J, Shazeer N, Hawthorne C, Dai AM, Hoffman MD, Eck D. An improved relative self-attention mechanism for transformer with application to music generation. CoRR arXiv:1809.04281 2018.
55. Wang Y, Lin Z, Shen X, Mech R, Miller G, Cottrell GW. Event-specific image importance. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) 2016.
56. Ren J, Shen X, Lin Z, Měch R. Best frame selection in a short video. In: 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), 2020;3201–3210. https://doi.org/10.1109/WACV45572.2020.9093615
57. Paszke A, Gross S, Chintala S, Chanan G, Yang E, DeVito Z, Lin Z, Desmaison A, Antiga L, Lerer A. Automatic differentiation in pytorch. 2017. https://api.semanticscholar.org/CorpusID:40027675
58. Min Y., et al.: Deep radial embedding for visual sequence learning. In: European Conference on Computer Vision. Springer, Cham 2022.
59. Papineni K, Roukos S, Ward T, Zhu W-J. Bleu: a method for automatic evaluation of machine translation. In: Annual Meeting of the Association for Computational Linguistics 2002. https://api.semanticscholar.org/CorpusID:11080756
60. Lin C-Y. Rouge: A package for automatic evaluation of summaries. In: Annual Meeting of the Association for Computational Linguistics 2004. https://api.semanticscholar.org/CorpusID:964287
61. Stojkovic J, Zhang C, Goiri Torrellas J, Choukse E. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency 2024. arXiv:2408.00741
62. Camgöz NC, Koller O, Hadfield S, Bowden R. Multi-channel transformers for multi-articulatory sign language translation. ArXiv arXiv:2009.00299 2020.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com